

Genesis CPO Voice AI Agent

Task Writeup · Fatima Obaid

Approach

The Genesis CPO site is built on Next.js and fully client-side rendered, so BeautifulSoup alone couldn't get the data. I used Playwright to drive a headless Chromium browser, paginate through each model filter (G80, G90, GV80), visit every individual listing page, and extract the fields that matter: year, trim, engine, price, mileage, colors, and features. That all goes into *inventory.json*.

For the RAG layer, I convert each car into a natural language string, something like "2025 Genesis GV80 3.5T Royal AWD, SAR 290,000, Capri Blue exterior, panoramic sunroof, heads-up display..." and embed it using sentence-transformers/all-MiniLM-L6-v2 into a local ChromaDB collection. When a customer asks a question, I embed their query the same way and pull the top 5 closest matches by cosine similarity. Those get injected into the LLM prompt as context, so the agent only answers based on what's actually in inventory.

The voice loop runs like this: sounddevice captures mic input with auto-stop on silence, Whisper transcribes it locally, Groq's Llama-3.3-70b generates a response grounded on the retrieved context, and ElevenLabs converts it to speech. I also built a fallback chain, if ElevenLabs fails, it falls back to gTTS, then pyttsx3, so the agent always has audio output.

Choices & Trade-offs

Whisper (local) over cloud STT: Running Whisper locally avoids an extra API hop in the voice loop and keeps transcription free. The trade-off is ~500ms CPU inference, switching to the small model improves accuracy at the cost of speed.

Groq over OpenAI: Groq's inference speed is the main reason. For voice, every extra second of latency is noticeable, and Groq consistently gives responses in under 500ms for this workload.

ChromaDB over Pinecone/Weaviate: ChromaDB runs fully in-process with no external service to set up or pay for. For a demo with 56 listings this is fine. If this were going to production with thousands of listings and concurrent users, I'd switch to a managed vector store.

RMS silence detection over VAD: Simple to implement and works well in a quiet room. In noisy environments it can cut off early, proper VAD (like Silero) would be more robust.

What I'd Improve With More Time

Streaming TTS: Right now the agent waits for the full LLM response before sending it to TTS. Streaming tokens directly into ElevenLabs' WebSocket API would cut time-to-first-audio from ~1.5 seconds down to under 300ms, a big difference in voice UX.

Barge-in handling: If the customer starts talking while the agent is still speaking, right now nothing happens. A proper implementation would run VAD continuously in parallel with TTS playback, detect the interruption, cut the audio stream, and restart the pipeline with the new input.

Better silence detection: The current RMS threshold is simple and can be thrown off by background noise. Silero VAD or WebRTC VAD would be more reliable.

Scheduled scraping: The scraper runs once and produces a static inventory.json. A scheduled Playwright job that refreshes inventory incrementally would keep the RAG layer current without re-indexing everything from scratch.

Assumptions

All prices on the site are in Saudi Riyals (SAR). I assumed all listed cars are available for purchase, the site shows listings without a clear sold/unavailable status on most, so I treated everything in inventory as active stock.